

TradeOnboarding Agent: 무역 실무 온보딩 AI 에이전트 분석 보고서

1. 프로젝트 개요 및 목적

TradeOnboarding Agent는 선박 기반의 수출입 무역회사에 새로 입사한 신입사원이 실무에 빠르게 적응할 수 있도록 돕는 **AI** 에이전트 기반 온보딩 시뮬레이터입니다. 이 시스템은 단순한 정보 전달을 넘어, 업스테이지(Upstage)의 **Solar API**를 활용하여 실제 업무 상황을 시뮬레이션하고 피드백을 제공함으로써 업무 숙련도를 높이는 데 주력합니다. 비즈니스 현장에서 발생하는 비효율을 기술적으로 해결하고자 하는 철학이 반영되어 있으며, 특히 무역 도메인의 복잡한 용어와 프로세스를 에이전트 환경에서 체득할 수 있도록 설계되었습니다.

2. 주요 기능 및 에이전트 구성

이 시스템은 기능별로 특화된 **4개의 핵심 에이전트**로 구성되어 있으며, 사용자의 의도에 따라 적절한 에이전트가 호출됩니다. | 에이전트 명칭 | 주요 역할 및 기능 || ----- | ----- ||  **Quiz Agent** | 무역 용어 및 실무 프로세스에 관한 퀴즈를 생성하고 사용자의 답변을 평가 ||  **Email Coach Agent** | 신입사원이 작성한 이메일에 대한 피드백을 제공하고 비즈니스 매너 코칭 ||  **Mistake Predictor Agent** | 무역 실무 상황별로 발생할 수 있는 실수를 예측하고 예방 가이드 제시 ||  **CEO Simulator Agent** | 대표(CEO)에게 보고하는 상황을 가정하여 시뮬레이션 및 연습 기회 제공 |

3. 기술 스택 및 시스템 아키텍처

3.1 기술 스택 (Tech Stack)

현대적인 **Python** 생태계의 도구들을 활용하여 빠르고 효율적인 개발 환경을 구축하였습니다.

- **Backend:** FastAPI (웹 프레임워크), LangChain (에이전트 프레임워크)
- **LLM & Embedding:** Upstage Solar API, Upstage Solar Embedding
- **Vector Store:** ChromaDB (RAG 시스템 구축용)
- **Package Manager:** uv (고속 Python 패키지 매니저)
- **Frontend:** Streamlit (Python 기반 UI 프레임워크)

3.2 시스템 아키텍처 및 로직

시스템은 **인텐트 라우팅(Intent Routing)** 로직을 따르는 오케스트레이터(Orchestrator)를 중심으로 동작합니다.

- **에이전트 라우팅 로직:** 사용자의 입력 메시지를 분석하여 키워드에 따라 적절한 에이전트로 연결합니다.
- "퀴즈", "quiz", "문제" → **Quiz Agent**
- "메일", "email", "이메일" → **Email Agent**
- "보고", "대표", "CEO" → **CEO Simulator**
- "실수", "mistake", "주의" → **Mistake Predictor**
- 그 외 일반적인 질문 → **General Q&A**
- **RAG 시스템:** 무역 도메인 지식을 ChromaDB에 저장하고 필요시 검색(Retriever)하여 답변의 정확도를 높입니다.

4. 데이터셋 및 지식 베이스

시스템은 실무적인 학습을 위해 **200개** 이상의 정형화된 데이터 포인트를 보유하고 있습니다.

- 기업 도메인 지식: 회사 내부 프로세스, 도메인 전문 지식 각 **20**항목.
- 실무 케이스: 실수 사례(총 **40**항목), 이메일 로그(**20**항목), 국가별 규칙(**20**항목).
- 커뮤니케이션: 대표 결정 스타일 및 시뮬레이션 데이터(**40**항목), 협상 및 클레임 사례(**40**항목).
- 기타: 무역 **Q&A**, **KPI** 데이터, 퀴즈 샘플 등.

5. 프로젝트 구조 및 주요 모듈

```
trade-onboarding-agent/
├── backend/
│   ├── main.py           # FastAPI 애플리케이션 진입점
│   └── agents/           # 에이전트 구현체 (orchestrator, quiz,
email 등)
│       ├── rag/         # RAG 시스템 (retriever, vector data)
│       ├── prompts/     # LLM용 프롬프트 템플릿 파일
│       └── api/         # API 엔드포인트 정의
├── frontend/
│   └── app.py           # Streamlit 기반 사용자 인터페이스
├── dataset/             # 시뮬레이션용 데이터셋 (JSON 및 Markdown)
├── pyproject.toml       # uv 프로젝트 설정
└── .env                # API 키 등 환경 변수 관리
```

6. 개발 로드맵 및 현황

프로젝트는 **4일**간의 집중 개발 과정을 통해 구축되었습니다.

1. **Day 1:** 프로젝트 구조 생성, **FastAPI** 및 **Streamlit** 기본 세팅, **uv** 기반 패키지 관리 설정.
2. **Day 2 ~ 3:** 4종의 핵심 에이전트(Quiz, Email, Mistake, CEO) 기능 구현.
3. **Day 3** 오후: 에이전트 통합(Orchestrator), RAG 시스템 구현 및 **ChromaDB** 연동.
4. **Day 4:** 관리 대시보드 구현, 최종 테스트 및 배포 준비.

7. 분석가 통찰: 비즈니스 가치

이 프로젝트는 단순히 기술적인 구현에 그치지 않고, *****현장의 결핍*****에서 시작된 문제 해결 의지를 담고 있습니다. 개발자는 **10년간**의 비즈니스 현장 경험을 통해 얻은 '효율에 대한 갈증'을 **AI** 기술로 풀어내고자 하였습니다.

- 실질적 문제 해결: 감에 의존하는 교육이 아닌, 정확한 데이터와 프로세스 설계(**Process Design**)를 통해 신입사원의 병목 현상을 해결합니다.
- 몰입도 높은 기술 적용: 메카트로닉스 공학적 사고와 비즈니스 실무 경험을 결합하여, 실제 현장에서 즉시 적용 가능한 솔루션을 지향합니다.
- 성장 지향적 구조: **LLM**과 **RAG** 기술을 활용하여 지식이 휘발되지 않고 자산화될 수 있도록 시스템을 구축하였습니다."저는 단순히 성능 좋은 모델을 만드는 것에 그치지 않고, 이 기술이 실제 비즈니스 현장의 어떤 문제를 해결할 수 있는지 고민합니다." (Source Context 인용)